



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

大数据计算及应用实验报告

Lab1 PageRank

2212574 文雅竹

2312966 林晖鹏

2313725 张耕嘉

指导教师：杨征路

2025 年 4 月 29 日

摘要

本实验通过实现并优化 PageRank 算法, 分析了不同优化策略在内存使用和运行效率上的表现。实验基于包含 9500 个节点和 15 万条边的数据集, 重点解决了死胡同和蜘蛛陷阱问题, 并采用稀疏矩阵、文件读取优化及分块矩阵等方法优化内存。结果表明: 未优化的基础算法内存占用高达 776.7 MB, 运行时间为 2.96 秒; 通过稀疏矩阵优化, 内存降至 **9.36 MB**, 运行时间缩短至 **0.20 秒**; 进一步的文件读取优化将内存压缩至 **2.56 MB**, 时间保持稳定; 而分块矩阵优化内存为 **3.25 MB**, 但运行时间显著增加至 **39.76 秒**。实验验证了稀疏矩阵优化在内存效率与计算速度上的显著优势, 同时揭示了分块策略在时间成本上的局限性, 为大规模图数据处理提供了有效的优化方向。

关键字: PageRank 算法; 稀疏矩阵优化; 块矩阵优化; 内存优化; 大数据处理

目录

一、 实验内容	1
二、 问题描述	1
(一) 实验背景	1
(二) 实验目的	1
三、 实验原理	2
(一) PageRank 算法	2
(二) 死胡同 (dead ends)	2
(三) 蜘蛛陷阱 (Spider traps)	3
四、 实验过程	4
(一) Data 数据集	4
(二) 基本的 PageRank 算法	4
(三) 稀疏矩阵优化	6
(四) 文件读取优化	8
(五) 块矩阵优化	10
五、 实验结果	12
(一) 实验结果展示	12
1. 部分输出结果	12
2. 不同优化的性能展示	13
(二) 结果分析	13
六、 总结	13

一、 实验内容

计算给定数据集的 PageRank 分数

- 数据集: Data.txt
- 数据集文件中各行的格式如下: 从节点 ID 到节点 ID
- 在从这个项目中, 您需要报告前 100 个 NodeID 及其 PageRank 得分。您可以选择不同的参数 (例如 teltpot 参数) 来比较不同的结果。
- 必须报告的一个结果是, 将传送参数设置为 0.85 时。
- 除了基本的 PageRank 算法之外, 您还需要通过块矩阵、稀疏矩阵和其他一些方法来优化内存使用。

具体要求:

- 考虑死胡同和蜘蛛陷阱。
- 尽可能优化内存使用。确保程序整个生命周期内的最大内存使用量低于 80MB。程序不能牺牲太多的时间性能来最小化您的内存使用量。程序需要在 60 秒内完成运行。可以通过其他方式优化内存使用, 但块矩阵和稀疏矩阵优化是必须的。
- 程序需要迭代直到达到收敛。
- 禁止使用现有的 PageRank API。
- 程序需要读取 Data.txt, 进行 PageRank 计算, 并返回 Res.txt 中的 Top-100 个节点及其分数, 格式如下: 节点 ID 分数。

二、 问题描述

(一) 实验背景

随着互联网的飞速发展, 网页数量呈爆炸式增长, 如何从海量网页中筛选出重要信息成为关键问题。Pagerank 算法应运而生, 它通过分析网页间的链接结构来评估网页的重要性, 为搜索引擎排序等应用提供了重要依据。然而, 该算法在实际应用中存在蜘蛛陷阱 (网页所有出链接都在自身群组内, 导致随机游走被困, 重要性被吸收)、死链接 (网页没有出链接, 使随机游走无处可去, 重要性泄漏) 等问题, 影响了其准确性和有效性。

(二) 实验目的

深入理解 Pagerank 算法的原理, 包括其基于链接投票的核心思想、随机游走的概念以及矩阵计算方式; 分析 Pagerank 算法在实际应用中存在的问题, 如蜘蛛陷阱和死链接对算法结果的影响, 并探索有效的解决方案; 通过实验对比不同算法改进策略下的 PageRank 计算结果, 评估算法的性能和稳定性。

三、 实验原理

(一) PageRank 算法

PageRank 算法 [1] 基于这样一种思想，即一个网页的重要性不仅取决于它自身的内容和质量，还取决于其他网页对它的链接情况。如果一个网页被许多其他重要的网页链接到，那么它就被认为是重要的网页。

该算法将网页之间的链接视为一种投票行为。当一个网页 A 链接到另一个网页 B 时，就相当于网页 A 给网页 B 投了一票，这表明网页 A 认为网页 B 是有价值的，从而增加了网页 B 的重要性。当一个质量高的网页指向另一个网页，则这个被指向的网页也重要。

计算方法为，初始假设所有网页的初始 PageRank 值都相同，通常设为一个较小的常数，比如 $1/N$ 。然后通过不断迭代计算来更新每个网页的 PageRank 值，直到算法收敛，即 PageRank 值的变化小于某个阈值。用公式表示为

$$r_k = Mr_{k+1} \quad (1)$$

其中 r 为 PageRank 向量， M 是转移矩阵。元素 $M[i][j]$ 表示页面 j 到 i 的转移概率。

(二) 死胡同 (dead ends)

如果一个页面是死胡同，用户在浏览到该页面后无法通过点击链接继续访问其他页面，导致其 PageRank 值无法向下传递，如图1所示：

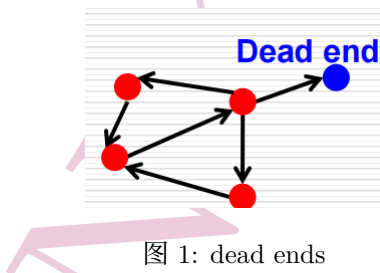


图 1: dead ends

在模拟图中可以看到，导致死胡同的节点就像黑洞一样将所有的用户访问吸入，导致其他节点无法再被访问。

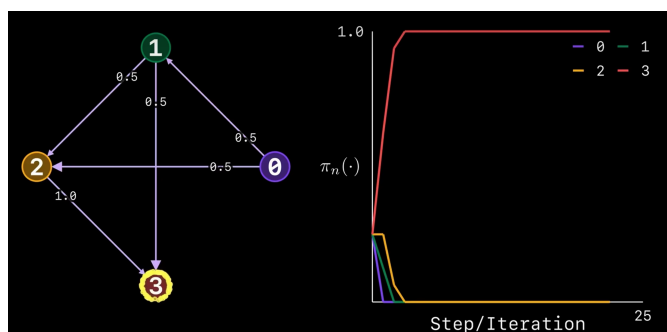
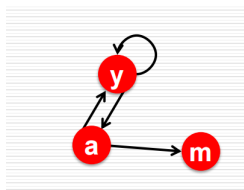


图 2: simulate [3]

转移矩阵 M 中，死胡同对应的列全为 0（因为它没有出链），导致矩阵不再是随机矩阵（列和不为 1）。在迭代计算中，系统的总权重会逐渐“流失”，最终所有 PageRank 值趋近于 0，例如：



	y	a	m
y	$\frac{1}{2}$	$\frac{1}{2}$	0
a	$\frac{1}{2}$	0	0
m	0	$\frac{1}{2}$	0

图 3: 示例图

$$\begin{pmatrix} r_y \\ r_a \\ r_m \end{pmatrix} = \begin{array}{c|cccccc} & 0\text{轮} & 1\text{轮} & 2\text{轮} & 3\text{轮} & \dots & \text{final} \\ \hline r_y & \frac{1}{3} & \frac{2}{6} & \frac{3}{12} & \frac{5}{24} & \dots & 0 \\ r_a & \frac{1}{3} & \frac{1}{6} & \frac{2}{12} & \frac{3}{24} & \dots & 0 \\ r_m & \frac{1}{3} & \frac{1}{6} & \frac{1}{12} & \frac{2}{24} & \dots & 0 \end{array}$$

对于死胡同问题，一个解决办法是修正转移矩阵（处理悬挂节点），通过强制分配权重，将所有死胡同页面视为“链接到所有页面”（包括自己）。具体的数学实现表现为，在转移矩阵 M 中，若页面是死胡同（无出链），则将其对应的列设为 $1/N$ 。这也可以理解为，当出现死胡同时，节点会百分之百随机传送。

经过该方法处理，PageRank 公式可修正为 [2]：

$$r_k = \left[M + \mathbf{a}^T \left(\frac{\mathbf{e}}{n} \right) \right] r_{k+1} \quad (2)$$

- $\mathbf{a} = [a_0, a_1, \dots, a_n]$ ，当有一列全为 0 时， $a_i = 1$ ，其他时候 $a_i = 0$
- $\mathbf{e}$ ：由 1 填满的列矩阵
- n ： M 矩阵的行数/列数

(三) 蜘蛛陷阱 (Spider traps)

蜘蛛陷阱 (Spider Traps) 是指一类具有特殊结构的网页群组，这些页面通过大量内部链接互相指向，但极少或没有指向外部页面的链接，如图所示：

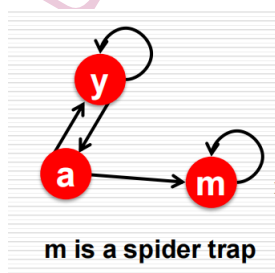


图 4: spider trap

这种结构会导致 PageRank 值在内部循环中不断累积，破坏权重的合理分配，最终使算法结果失真。

蜘蛛陷阱内的页面通过内部链接反复传递 PageRank 值，导致其权重被过度放大。

例如：若页面 A 只链接到 B，B 只链接到 C，C 又只链接回 A，则三者的 PageRank 值会在迭代中无限增长。若未引入阻尼因子，最终仅蜘蛛陷阱内的页面拥有非零 PageRank 值，其他页面的权重趋近于 0。如模拟所示，用户访问随迭代次数增加，只在蜘蛛陷阱内部流转，外部节点无法得到访问。

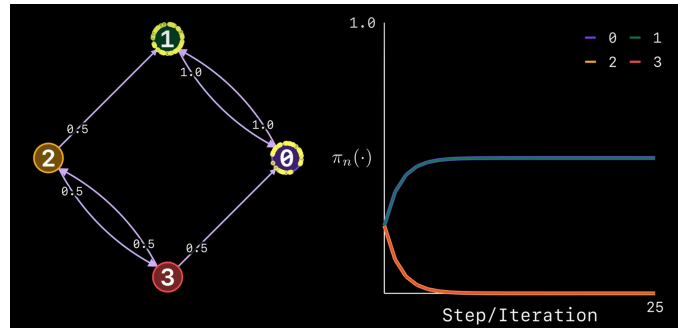


图 5: simulate [3]

蜘蛛陷阱的解决依赖于阻尼因子的引入，其核心思想是模拟用户随机跳转行为，避免权重无限循环。修正后公式为：

$$A = \beta M + (1 - \beta) \left[\frac{1}{N} \right]_{N \times N} \quad (3)$$

$$r = Ar \quad (4)$$

四、 实验过程

(一) Data 数据集

- 文件中各行的格式如下：从节点 ID 到节点 ID
- 数据集包含 9500 个节点
- 数据集包含 150000 条边

由于该数据集体量较大，所构建的矩阵巨大，为了让运行时间、占用内存存在要求范围之内，需要对时间复杂度、空间复杂度进行优化。

(二) 基本的 PageRank 算法

build_graph 函数从 Data.txt 文件读取图数据，逐行解析每对节点构成的边，忽略不符合格式的行。通过 set 结构收集所有唯一节点，确保无重复，随后将节点排序并建立从节点到唯一索引的映射字典。最终返回边列表、排序后的节点列表及索引字典，为后续矩阵操作提供基础数据结构。

build_graph

```

1  def build_graph(file_path):
2      edges = []
3      nodes = set()
4      with open(file_path, 'r') as f:
5          for line in f:
6              parts = line.strip().split()
7              if len(parts) != 2:
8                  print("Invalid line format:", line)
9                  continue
10             u, v = parts

```

```

11         edges.append((u, v))
12         nodes.update([u, v])
13     nodes = sorted(list(nodes))
14     node_index = {node: idx for idx, node in enumerate(nodes)}
15     return edges, nodes, node_index

```

build_transition_matrix 函数构建状态转移矩阵 M。函数接收边列表、节点列表和索引字典，构造一个表示节点间转移概率的方阵。首先根据边的存在性在矩阵对应位置标记 1，随后逐行处理：对每行的非零元素求和，若存在出边则归一化为概率分布（1/总出边数）；若行和为 0（即悬挂节点），则将该行设为均匀概率分布（每个节点概率相同，值为 1/N），通过这种方式处理死胡同问题。最终返回转置后的矩阵，使得矩阵乘法的方向与 PageRank 公式中“列表示目标节点”的定义一致，便于后续计算。

```

                                build_transition_matrix
1  def build_transition_matrix(edges, nodes, node_index):
2      N = len(nodes)
3      M = np.zeros((N, N))
4      for u, v in edges:
5          i = node_index[u]
6          j = node_index[v]
7          M[i, j] = 1
8      for i in range(N):
9          row_sum = M[i].sum()
10         if row_sum != 0:
11             M[i] /= row_sum
12         else:
13             M[i] = np.ones(N) / N
14     return M.T

```

pagerank 函数通过迭代更新概率向量实现 PageRank 计算。初始化时赋予所有节点相同概率，定义阻尼系数和随机跳转向量以模拟用户行为。函数使用公式

$$A = \beta M + (1 - \beta) \left[\frac{1}{N} \right]_{N \times N} \quad (5)$$

$$r = Ar \quad (6)$$

计算 pagerank，根据矩阵乘法分配率，由于 $(1 - \beta) \left[\frac{1}{N} \right]_{N \times N} r$ 是常量矩阵，命名为 teleport 拆出来单独计算。每次迭代中，新概率向量由两部分组成：阻尼系数乘以转移矩阵与原向量的乘积（模拟用户跟随链接跳转），以及（1-阻尼系数）乘以均匀跳转概率（模拟用户随机访问任意节点）。

通过计算新旧向量的 L1 范数差判断收敛性，若在容忍度内则提前终止迭代，否则持续更新直至达到最大迭代次数，最终返回稳定概率分布。

```

                                pagerank
1  def pagerank(M, damping=0.85, tol=1e-6, max_iter=10000):
2      N = M.shape[0]
3      pr = np.ones(N) / N
4      teleport = np.ones(N) / N
5      for _ in range(max_iter):
6          pr_new = damping * (M @ pr) + (1 - damping) * teleport

```

```

7         if np.linalg.norm(pr_new - pr, ord=1) < tol:
8             print(f"迭代第{iter}次之后, PageRank收敛")
9             return pr_new
10        pr = pr_new
11    return pr

```

对基本的 PageRank 算法进行内存分析, 如下所示。

Line #	Mem usage	Increment	Occurrences	Line Contents
60	57.3 MiB	0.0 MiB	1	file_path = "Data.txt"
61	86.8 MiB	29.6 MiB	1	edges, nodes, node_index = build_graph(file_path)
62	775.5 MiB	688.6 MiB	1	M = build_transition_matrix(edges, nodes, node_index)
63	775.7 MiB	0.2 MiB	1	pr_values = pagerank(M)

根据内存分析的结果, 可以看到在执行 `build_transition_matrix()` 函数时, 内存使用量大幅增加, 从 57.3 MiB 跃升至 775.5 MiB, 增幅高达 688.6 MiB。这表明 `build_transition_matrix()` 函数在计算转移矩阵时, 消耗了大量的内存。此时, 程序可能处理了过多的数据或采用了不必要的复杂操作, 从而导致内存占用过高。

另外, 文件读取的逻辑发生在 `build_graph()` 函数内, 它的内存增量也较大, 从 57.3 MiB 增加到 86.8 MiB, 说明在处理图数据时, 文件读取过程可能存在一定的内存消耗。虽然这个增量相对较小, 但依然需要优化。

因此, 可以推测, 内存占用的主要瓶颈来自转移矩阵的构建过程及文件的读取过程。为了提高程序的性能, 应该优化这两个部分:

- **转移矩阵构建:** 可以考虑使用稀疏矩阵 (如 CSR 格式) 来表示转移矩阵, 减少内存的消耗。稀疏矩阵在存储上仅存储非零元素, 能够显著减少内存的占用。
- **文件读取:** 可以通过优化文件读取方式, 如逐行读取或分批读取数据来避免一次性将整个文件加载到内存中, 减少内存峰值。

通过这些修改, 可以显著降低内存使用, 并提高程序的运行效率。

(三) 稀疏矩阵优化

PageRank 算法的关键步骤之一是计算转移矩阵 M 和 PageRank 向量 r 的乘积。在 PageRank 图的表示中, 节点之间的连接 (边) 通常是稀疏的, 也就是说, 大多数节点并不直接连接到其他节点。稀疏矩阵正是为了解决这种问题而设计的。普通的密集矩阵 (即每个元素都占用内存) 在图非常大的时候会非常低效, 因为大部分元素都是零, 浪费了大量内存。

稀疏矩阵乘法 `@` 的好处在于:

- **计算效率:** 通过只计算非零元素的乘积, 避免了对整个矩阵进行操作。例如, 在稀疏矩阵中, 只有图中有边的节点对会产生非零乘积, 其他部分的乘积可以跳过不计算, 这大大提高了乘法的效率。
- **内存效率:** 稀疏矩阵的乘法只存储和处理非零元素, 避免了密集矩阵乘法中大部分无效的零元素操作。

通过上文中的方法修正转移矩阵后，对于所有的死胡同，其对应的列的所有值都相同，因此可以将所有的死胡同单独在一个一维数组 *death* 中被标记出来，并在 PageRank 计算中加以处理：对于死节点，它的 PageRank 值会均匀地分配到所有节点，因此通过将这个一维数组与向量 *r* 相乘，添加死节点的影响，再添加到整体的结果中，作为新的 *r* 向量。这样将转移矩阵分为两部分：*M* 和用一维向量表示的 *death*，减小内存占用。

$$\mathbf{r}_{\text{new}} = \beta \times (\mathbf{M} \cdot \mathbf{r}) + \frac{\beta}{N} \times (\mathbf{death} \cdot \mathbf{r}) + (1 - \beta) \times [\frac{1}{N}]_{N \times N}$$

构建稀疏矩阵和 death 数组

```

1 def build_transition_matrix(edges, nodes, node_index):
2     N = len(nodes)
3     # 使用COO格式初始化稀疏矩阵（适合构建阶段）
4     row_indices = []
5     col_indices = []
6     data = []
7
8     # 统计每个节点的出度
9     out_degrees = np.zeros(N)
10    for u, v in edges:
11        i = node_index[u]
12        j = node_index[v]
13        out_degrees[i] += 1
14
15    # 构建稀疏矩阵的数据
16    for u, v in edges:
17        i = node_index[u]
18        j = node_index[v]
19        if out_degrees[i] > 0:
20            row_indices.append(j) # 转置后的行（原列）
21            col_indices.append(i) # 转置后的列（原行）
22            data.append(1.0 / out_degrees[i])
23
24
25    death = np.zeros(N, dtype=np.float32)
26    # 处理死节点（没有出边的节点）
27    for i in range(N):
28        if out_degrees[i] == 0:
29            death[i] = 1
30
31    # 创建稀疏矩阵（已转置）
32    M = sparse.csr_matrix((data, (row_indices, col_indices)), shape=(N, N))
33    return M, death

```

优化 PageRank 函数

```

1 def pagerank(M, death, damping=0.85, tol=1e-6, max_iter=10000):
2     N = M.shape[0]
3     # 初始化概率向量

```

```

4 pr = np.ones(N, dtype=np.float32) / N
5 teleport = np.ones(N, dtype=np.float32) / N
6
7 for iter_count in range(max_iter):
8     # 稀疏矩阵乘法
9     pr_new = damping * (M @ pr) + damping/N * (death @ pr) + (1 - damping)
10         * teleport
11
12     # 计算收敛程度
13     err = np.linalg.norm(pr_new - pr, ord=1)
14     pr = pr_new
15
16     if err < tol:
17         print(f"迭代第{iter_count}次之后, PageRank_收敛")
18         return pr
19
20 print(f"警告: 达到最大迭代次数{max_iter}, 但未收敛")
21 return pr

```

(四) 文件读取优化

在稀疏矩阵优化的基础上, 我们从内存空间有效利用率的角度, 对先前的优化进行进一步优化。可以从以下几个角度优化:

1. stl 内存管理

我们知道, stl 如果没有预设置空间大小, 一旦越界, 会创造多一倍的空间, 这样会导致预留的空间多了很多没有使用的。结合读取文件对于时间内存花销并不是很大, 我们预先读取文件获取边的个数, 用来预先设置存边的容器的大小, 来节省不必要的空间。

2. 减少重复信息读取

一开始, 我们读取了稀疏矩阵的边信息, 但是在构建稀疏矩阵的三元组的时候, 我们又重新储存了一次边信息。这里重复存了两次边的内存, 这里我们改成线读取出度信息, 在读取三元组的时候在读取边信息。

以下是主要的优化步骤:

1. 首先, 估计边数

```

1 file.seekg(0, std::ios::end);
2 size_t file_size = file.tellg();
3 file.seekg(0, std::ios::beg);
4
5 while (std::getline(file, line0))
6 {
7     line_count++;
8 }

```

2. 接下来, 收集节点出度, 并构建节点索引信息。

```

1 while (std::getline(file , line))
2 {
3     std::istringstream iss(line);
4     if (!(iss >> u >> v))
5     {
6         std::cerr << "Invalid_line_format:" << line << std::endl;
7         continue;
8     }
9     node_set[u] = true;
10    node_set[v] = true;
11 }
12 for (const auto &pair : node_set)
13 {
14     nodes.push_back(pair.first);
15 }
16 std::sort(nodes.begin(), nodes.end());
17 for (int i = 0; i < nodes.size(); i++)
18 {
19     node_index[nodes[i]] = i;
20 }
21
22 out_degrees.resize(nodes.size(), 0);
23 while (std::getline(file , line))
24 {
25     std::istringstream iss(line);
26     if (!(iss >> u >> v))
27     {
28         std::cerr << "Invalid_line_format:" << line << std::endl;
29         continue;
30     }
31     int u_id = node_index.at(u);
32     out_degrees[u_id]++;
33 }

```

3. 随后，预设置三元组大小，并且读取边信息，构建稀疏矩阵的三元组

```

1 triplets.reserve(line_count);
2 while (std::getline(file , line))
3 {
4     std::istringstream iss(line);
5     if (!(iss >> u >> v))
6     {
7         std::cerr << "Invalid_line_format:" << line << std::endl;
8         continue;
9     }
10    int u_id = node_index.at(u);
11    int v_id = node_index.at(v);
12    if(out_degrees[u_id] > 0)

```

```

13     {
14         // 注意：矩阵已转置 - j 行 i 列
15         triplets.push_back(std::make_tuple(v_id, u_id, 1.0f / out_degrees[u_id]));
16     }
17     // triplets.push_back(std::make_tuple(u_id, v_id, 1.0f / out_degrees[u_id]));
18 }

```

通过这些优化步骤，我们能够高效地从文件中读取并构建图数据，避免了在内存中重复存储整个图的数据结构以及不必要的内存空间浪费，从而提高了程序的性能。

这里我们还尝试了去预设节点容器的大小，但是结果反而更差。

（五） 块矩阵优化

将图的节点划分为多个部分（块），间接对矩阵进行分块，并对每个部分的稀疏矩阵进行独立计算。这种方法类似于传统的**块矩阵（Block Matrix）**处理技术，可以提高大规模稀疏图计算的效率，特别是在并行计算的场景下。具体实现步骤如下：

1. 图的分块：首先，我们将图的节点划分为多个部分，每个部分包含一段连续的节点。例如，将节点从 0 到 N/num 划分为一个部分，第二部分是从 N/num 到 $2N/num$ ，依此类推。
2. 独立计算每个子矩阵的 PageRank：对于每个节点部分，我们构建对应的稀疏矩阵，并进行矩阵-向量乘法操作，即计算该部分的 PageRank 值。这部分计算是独立的，并且只涉及该部分的非零元素，避免了对整个图的计算。具体实现如下：

$$r_{part} = \beta \times (M_{part} \times r)$$

3. 合并结果：每个部分的计算结果 (r_{new_part}) 会被逐元素加到全局的 r_{new} 向量中，从而得到最终的 PageRank 向量。这一过程类似于将每个子矩阵的计算结果合并到全局结果中，类似于块矩阵的合并操作。具体实现如下：

$$r[i + idx_{start}] += r_{part}[i]$$

具体的实现代码如下所示。

块矩阵 PageRank

```

1 std::vector<float> pagerank(const std::string &file_path,
2                             int parts_num,
3                             std::vector<float> &death,
4                             std::vector<std::string> &nodes,
5                             std::vector<int> &out_degrees,
6                             std::unordered_map<std::string, int> &
7                             node_index,
8                             float damping = 0.85f, float tol = 1e-6f, int
9                             max_iter = 10000)
10 {
11     int N = nodes.size();

```

```

10 // 初始化概率向量
11 std::vector<float> pr = VectorOps::constant(N, 1.0f / N);
12 std::vector<float> teleport = VectorOps::constant(N, 1.0f / N);
13 std::vector<float> pr_new = VectorOps::constant(N, 0);
14 std::cout<<1<<std::endl;
15
16 for (int iter_count = 0; iter_count < max_iter; iter_count++)
17 {
18     pr_new = VectorOps::constant(N, 0);
19     for(int part =0;part<parts_num;part++){
20         int idx_start = part * N / parts_num;
21         int idx_end = (part + 1) * N / parts_num - 1;
22         SparseMatrix M_part;
23         std::vector<std::tuple<int, int, float>> triplets;
24         triplets.clear();
25         build_graph_part(file_path, nodes, node_index, out_degrees, triplets,
26             M_part, idx_start, idx_end);
27         std::vector<float> pr_new_part;
28         pr_new_part.resize(idx_end-idx_start+1,0);
29         pr_new_part = VectorOps::multiply(M_part.multiply(pr), damping);
30         //逐元素加到pr_new
31         for(int i = 0; i < pr_new_part.size(); i++){
32             pr_new[i+idx_start] += pr_new_part[i];
33         }
34     }
35     std::cout << iter_count << std::endl;
36     // 添加死节点贡献: damping/N * (death · pr) * ones(N)
37     float death_contribution = damping / N * VectorOps::dot(death, pr);
38     std::vector<float> death_vector = VectorOps::constant(N,
39         death_contribution);
40     pr_new = VectorOps::add(pr_new, death_vector);
41
42     // 添加随机跳转: (1-damping) * teleport
43     std::vector<float> random_jump = VectorOps::multiply(teleport, 1.0f -
44         damping);
45     pr_new = VectorOps::add(pr_new, random_jump);
46
47     // 计算收敛误差
48     std::vector<float> diff = VectorOps::subtract(pr_new, pr);
49     float err = VectorOps::l1Norm(diff);
50     pr = pr_new;
51
52     if (err < tol)
53     {
54         std::cout << "迭代第" << iter_count << "次之后, PageRank_收敛" << std::
55             endl;
56         return pr;
57     }
58 }

```

```
54 }  
55  
56 std::cout << "警告：达到最大迭代次数" << max_iter << ", 但未收敛" << std::  
    endl;  
57 return pr;  
58 }
```

五、 实验结果

(一) 实验结果展示

1. 部分输出结果

节点 ID	PageRank 值	节点 ID	PageRank 值
286	0.00019801	4352	0.00018200
3473	0.00019580	7032	0.00018147
4951	0.00019250	7541	0.00018102
3890	0.00019106	3699	0.00018047
7365	0.00018896	4877	0.00017852
6359	0.00018761	3242	0.00017669
2276	0.00017398	6503	0.00017576
1189	0.00017397	4221	0.00017495
1441	0.00017158	7293	0.00017425
1866	0.00017076	2276	0.00017398
8602	0.00017030	1189	0.00017397

表 1: 节点与对应的 PageRank 值

如表 1所示，不同优化代码的计算结果均如上所示，计算结果正确。

2. 不同优化的性能展示

优化方式	最大内存使用 (MB)	运行时间 (秒)
无优化	776.7	2.96
稀疏矩阵优化	9.36	0.20
文件读入优化	2.56	0.21
分块矩阵优化	3.25	39.76

表 2: 不同优化方式下内存使用和时间消耗

(二) 结果分析

表 2展示了在不同优化方式下，内存使用和运行时间的对比。我们可以从表中得出以下几点结论：

- 在**无优化**情况下，最大内存使用量为 776.7MB，运行时间为 2.96 秒。这说明，未经优化的程序在内存消耗和执行时间方面都存在较大的负担。
- 稀疏矩阵优化**大幅减少了内存使用量，降低至 9.36MB，并且运行时间也显著缩短，为 0.20 秒，表明此优化方式极大地提升了效率。
- 文件读入优化**在稀疏矩阵优化的基础上进一步优化,将内存从 9.36MB 进一步降低为 2.56MB，同时时间花销几乎不变，进一步轻量化了 PageRank 算法。
- 分块矩阵优化**的内存使用为 3.25MB，运行时间则高达 39.76 秒，远远超过其他优化方式的时间，通过内存分析，此时主要内存为储存节点信息的数组以及计算过程中的内存花销。在 60s 的限制内，我们其实还可以进一步分割更多块进行迭代计算，进一步通过时间换空间来压缩内存使用。

综上所述，稀疏矩阵优化在减少内存使用和提高运行效率方面均表现最佳，而分块矩阵优化虽然减少了内存使用，但在运行时间上的表现不佳，可能需要进一步优化或调整策略。

六、 总结

本次大数据计算及应用实验旨在深入理解 PageRank 算法的原理及其在实际应用中的优化。通过对给定数据集 Data.txt 进行 PageRank 分数计算，我们探索了如何通过不同的优化方法提高算法的效率和内存使用率。以下是实验的关键发现与总结：

1. PageRank 算法概述

PageRank 算法基于网页链接投票机制，用于评估网页的重要性。在计算过程中，每个网页的 PageRank 值由所有指向它的网页的 PageRank 值决定。该算法通过矩阵乘法迭代计算，直到收敛为止。实验中通过实现基础的 PageRank 算法，并对其进行优化，考虑了死胡同和蜘蛛陷阱等问题。

2. 优化方法

- **稀疏矩阵优化**: 由于 PageRank 计算过程中涉及的矩阵是稀疏矩阵 (大部分元素为零), 因此采用稀疏矩阵格式 (如 CSR 格式) 大幅度减少内存消耗, 并提升计算效率。
- **文件读取优化**: 在稀疏矩阵优化的基础上, 减少了不必要的文件信息重复读取, 并显示设置 stl 容器的大小, 进一步减少内存使用以及内存浪费。
- **分块矩阵优化**: 将大矩阵分成多个块并独立计算, 这种方法有助于并行计算, 但在本实验中由于大规模矩阵分块处理, 运行时间有所增加。

3. 实验结果

实验结果展示了不同优化方式下的内存使用和运行时间对比, 具体结果如下:

- **无优化**: 最大内存使用为 776.7MB, 运行时间为 2.96 秒。
- **稀疏矩阵优化**: 内存使用降至 9.36MB, 运行时间缩短至 0.20 秒。稀疏矩阵优化显著提升了内存效率和执行速度。
- **文件读入优化**: 内存使用降至 2.56MB, 运行时间为 0.21 秒, 优化效果优于无优化情况。
- **分块矩阵优化**: 内存使用为 3.25MB, 但运行时间为 39.76 秒, 可能需要进一步的调整。

4. 结论

- **稀疏矩阵优化**是本次实验中最有效的优化方法, 在大幅减少内存使用的同时, 显著提升了计算速度。
- **文件读取优化**在稀疏矩阵优化的基础上, 进一步优化, 尽可能的压缩内存的使用, 同时不增加时间花销。
- **分块矩阵优化**尽管减少了内存占用, 但由于分块计算的开销, 反而导致了运行时间的增加, 可能需要进一步的优化策略。

通过本次实验, 深刻理解了 PageRank 算法的工作原理, 并成功应用了稀疏矩阵、文件读取优化和分块矩阵等方法。实验表明, 优化内存使用的同时, 必须考虑对运行时间的影响, 因此不同优化方法的结合使用至关重要。

参考文献

- [1] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The pagerank citation ranking: Bringing order to the web. Technical Report 1999-66, Stanford InfoLab, November 1999. Previous number = SIDL-WP-1999-0120.
- [2] weixin_40431584. 手把手图文并茂教你掌握 pagerank 算法. *CSDN 博客*, 09 2024.
- [3] 同济子豪兄. 改变世界的谷歌 pagerank 算法. 哔哩哔哩, 12 2022.

NIJU